

## DLT + Auto Loader (Function-wise) – FULL Q&A (with Code)

---

### ✓ A) DLT Basics & Architecture (1–10)

#### 1) What is Delta Live Tables (DLT)?

##### ✓ Answer:

DLT is a Databricks framework to build **reliable, declarative ETL pipelines** with built-in:

- ✓ lineage
  - ✓ monitoring
  - ✓ quality checks (expectations)
  - ✓ dependency handling
  - ✓ automatic table management
- 

#### 2) Why use DLT instead of normal notebooks?

##### ✓ Answer (Senior):

Notebooks are manual orchestration. DLT provides:

- automatic dependency graph
  - pipeline-managed execution
  - event logs + monitoring
  - expectations for data quality
- So production support becomes easier.
- 

#### 3) What is Medallion architecture in DLT?

##### ✓ Answer:

DLT supports Bronze → Silver → Gold pipeline design:

- Bronze = raw
  - Silver = cleaned
  - Gold = aggregated / reporting-ready
- 

#### 4) What are the main DLT objects?

##### ✓ Answer:

- @dlt.table (managed tables)
  - @dlt.view (logical views)
  - expectations (DQ rules)
  - pipeline settings (cluster/config)
- 

### 5) What is the difference between DLT table and view?

 **Answer:**

- dlt.table → physically stored Delta table
  - dlt.view → logical result used for intermediate logic
- 

### 6) What is a DLT pipeline?

 **Answer:**

A pipeline is the execution container that runs DLT code and manages:

- cluster
  - compute
  - table materialization
  - refresh scheduling
- 

### 7) DLT supports streaming and batch?

 **Answer:**

Yes:

- Streaming = spark.readStream
  - Batch = spark.read
- 

### 8) What is the purpose of Bronze layer in DLT?

 **Answer:**

To ingest raw data with minimal transformation and maintain traceability.

---

### 9) What is the purpose of Silver layer?

 **Answer:**

To apply data standardization, enrichment, validations, and ready data for analytics.

---

**10) What is the purpose of Gold layer?**

 **Answer:**

Gold layer provides final reporting-ready tables like KPI aggregates, fact/dim tables.

---

---

 **B) Auto Loader (cloudFiles) – Core Questions (11–22)**

**11) What is Auto Loader?**

 **Answer:**

Databricks Auto Loader incrementally ingests new files from cloud storage (ADLS/S3) using cloudFiles.

---

**12) Why Auto Loader over normal spark read?**

 **Answer:**

- incremental ingestion
  - efficient file discovery
  - scalable
  - schema evolution support
  - works in streaming mode
- 

**13) What is cloudFiles format?**

 **Answer:**

A special source in Spark:

`spark.readStream.format("cloudFiles")`

---

**14) How do you read Parquet using Auto Loader?**

 **Code:**

```
df = (spark.readStream
      .format("cloudFiles")
      .option("cloudFiles.format", "parquet")
      .load(path))
```

---

### 15) How do you read CSV using Auto Loader?

#### ✅ Code:

```
df = (spark.readStream
      .format("cloudFiles")
      .option("cloudFiles.format", "csv")
      .option("header", "true")
      .option("inferSchema", "true")
      .load(path))
```

---

### 16) What is schemaLocation?

#### ✅ Answer:

It stores schema metadata + checkpoint information used by Auto Loader.

#### ✅ Example:

```
.option("cloudFiles.schemaLocation", "/mnt/checkpoints/address")
```

---

### 17) Why schemaLocation should be unique per table?

#### ✅ Answer:

If multiple tables share one schemaLocation, schema mismatches happen and ingestion fails.

So best practice: **1 checkpoint per dataset.**

---

### 18) What is schemaEvolutionMode?

#### ✅ Answer:

Controls schema drift behavior in Auto Loader.

---

### 19) What is rescue mode?

✓ **Answer:**

When unexpected columns arrive, they are captured under a rescued column instead of failing pipeline.

✓ **Example:**

```
.option("cloudFiles.schemaEvolutionMode", "rescue")
```

---

### 20) What is the advantage of rescue mode?

✓ **Answer:**

Pipeline will not fail due to schema drift, and data is not lost.

---

### 21) How do you track file name in Auto Loader?

✓ **Code:**

```
from pyspark.sql.functions import col, current_timestamp
```

```
df = df.withColumn("_source_file", col("_metadata.file_path")) \
    .withColumn("_ingest_time", current_timestamp())
```

---

### 22) Auto Loader supports incremental ingestion how?

✓ **Answer:**

Auto Loader tracks processed files using checkpoint, so only newly arrived files are read.

---

---

✓ **C) DLT Expectations (Data Quality) (23–32)**

### 23) What are DLT expectations?

✓ **Answer:**

Expectations are data quality rules to validate data and control bad records.

---

## 24) Types of expectation actions?

✅ **Answer:**

- drop invalid rows
  - fail pipeline
  - quarantine (keep invalid records separately)
- 

## 25) Expectation example (not null)

✅ **Code:**

```
import dlt
```

```
@dlt.table
```

```
@dlt.expect("valid_id", "id IS NOT NULL")
```

```
def silver_customer():
```

```
    return dlt.read("bronze_customer")
```

---

## 26) Drop invalid records example

✅ **Code:**

```
@dlt.table
```

```
@dlt.expect_or_drop("valid_id", "id IS NOT NULL")
```

```
def silver_customer():
```

```
    return dlt.read("bronze_customer")
```

---

## 27) Fail pipeline if invalid

✅ **Code:**

```
@dlt.table
```

```
@dlt.expect_or_fail("valid_id", "id IS NOT NULL")
```

```
def silver_customer():
```

```
return dlt.read("bronze_customer")
```

---

## 28) Quarantine invalid records example

### Code:

```
@dlt.table
@dlt.expect_or_drop("valid_amount", "amount >= 0")
def silver_sales():
    return dlt.read("bronze_sales")
```

---

## 29) Why expectations are important?

### Answer:

They ensure data quality at pipeline level and reduce downstream issues in reports.

---

## 30) How do you capture expectation metrics?

### Answer:

DLT automatically logs expectation pass/fail metrics in pipeline event logs.

---

## 31) What validations can be applied?

### Answer:

- not null checks
  - unique checks
  - range checks
  - relationship checks
  - accepted values
  - duplicate removal
- 

## 32) How do you handle duplicates using expectations?

 **Answer:**

Expectations mainly validate, for dedup we apply window logic and keep latest record.

---

---

 **D) Streaming vs Batch in DLT (33–38)**

**33) Why use readStream in DLT for file ingestion?**

 **Answer:**

It supports incremental file loads and continuous processing using Auto Loader.

---

**34) How is streaming different from batch?**

 **Answer:**

Streaming processes data continuously in micro-batches and keeps state.

---

**35) Can we mix batch and streaming tables in DLT?**

 **Answer:**

Yes, DLT supports batch+streaming combined pipelines.

---

**36) What is a micro-batch?**

 **Answer:**

Small batches of data processed in streaming mode.

---

**37) How to write streaming data into Delta?**

 **Answer:**

DLT manages the write automatically when returning a streaming DF inside @dlt.table.

---

**38) What is checkpointing in streaming?**

 **Answer:**

Checkpoint stores progress so stream can resume safely after restart.

---

---



## ✅ E) Delta + Performance + Production (39–50)

### 39) What is Delta Lake?

#### ✅ Answer:

Delta is a storage format with:

- ACID transactions
  - time travel
  - merge/upsert
  - schema enforcement
- 

### 40) How do you do Upsert in Delta?

#### ✅ Answer:

Using MERGE (generally in Silver/Gold).

---

### 41) OPTIMIZE vs VACUUM?

#### ✅ Answer:

OPTIMIZE compacts small files.

VACUUM cleans old unused files.

---

### 42) How to avoid small file problem?

#### ✅ Answer:

- optimize tables
  - proper partitioning
  - avoid too many small writes
- 

### 43) What is ZORDER?

#### ✅ Answer:

Clustering technique to improve query performance for filtered columns.

---

### 44) How do you monitor DLT pipelines?

✓ **Answer:**

- pipeline UI status
  - event logs
  - expectation metrics
  - table refresh history
- 

**45) How do you handle pipeline failures?**

✓ **Answer:**

- check failed stage/activity
  - identify root cause (schema change/data issue)
  - fix mapping or rescue handling
  - rerun pipeline safely
- 

**46) How do you implement idempotency?**

✓ **Answer:**

Use MERGE for incremental, and partition overwrite for daily loads.

---

**47) What is the best way to do incremental load in DLT?**

✓ **Answer:**

Auto Loader for raw ingestion + MERGE in Silver/Gold.

---

**48) How do you implement audit columns?**

✓ **Answer:**

Add default audit columns in bronze ingestion (like you did).

✓ **Example:**

```
df = df.withColumn("recordCreatedOn", current_timestamp())
```

**49) Why include `_source_file` and `_ingest_time`?**

✓ **Answer:**

To support traceability, debugging, and audit requirements.